

El proyecto Fénix:

Mejora del código heredado (RA4b)



Este documento recoge el desarrollo completo de la actividad de optimización de código Java utilizando herramientas del IDE: control de versiones, análisis estático, refactorización, documentación e integración continua.

1. CONFIGURACIÓN DEL PROYECTO EN GIT

Tarea 1: Control de versiones local y remoto

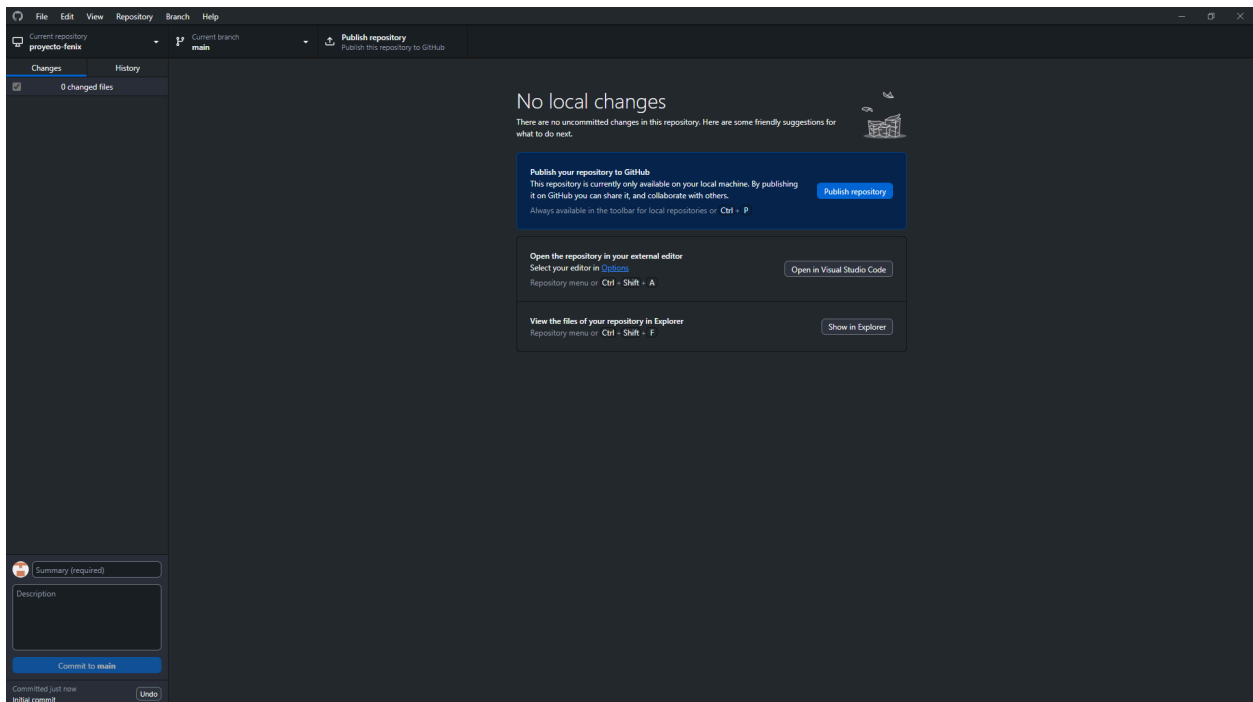
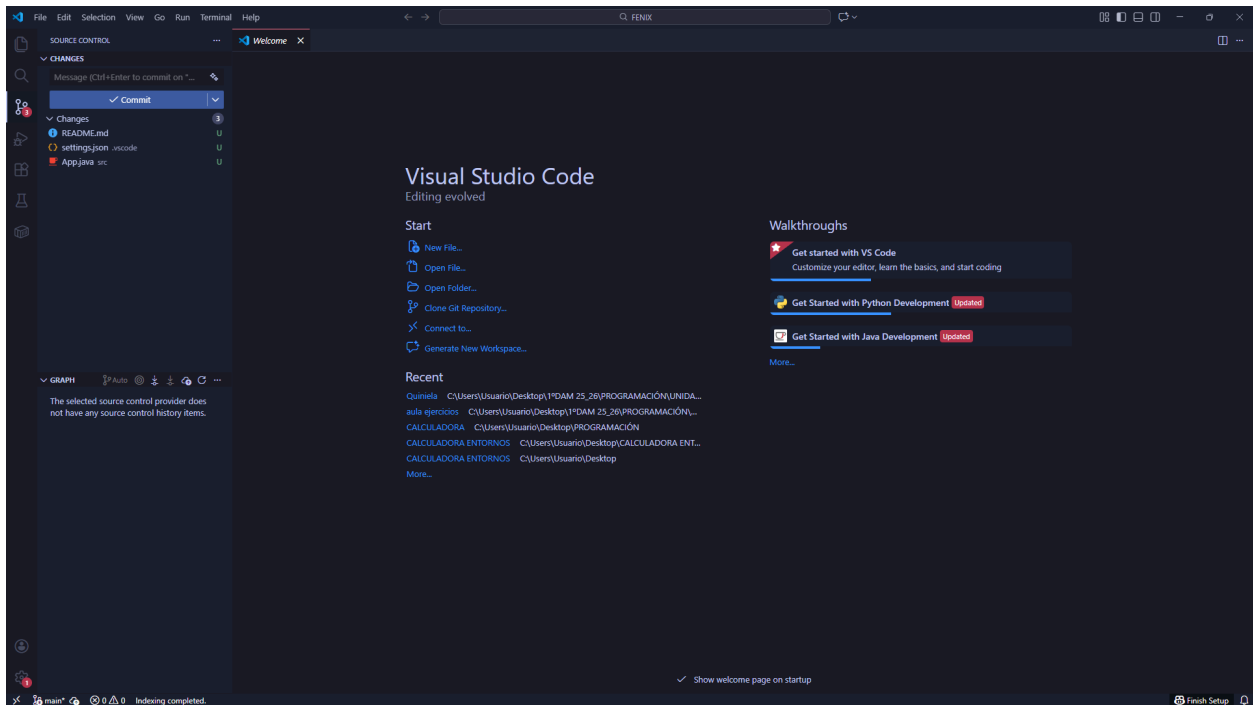
1. Se crea un nuevo proyecto Java en el IDE e incluye la clase `ProcesadorUsuarios.java`.
2. Desde el panel Git del IDE se inicializa un repositorio local.
3. En GitHub se crea un repositorio remoto vacío llamado proyecto-fénix.
4. Se conecta el repositorio local con el remoto y se realiza el commit inicial con el

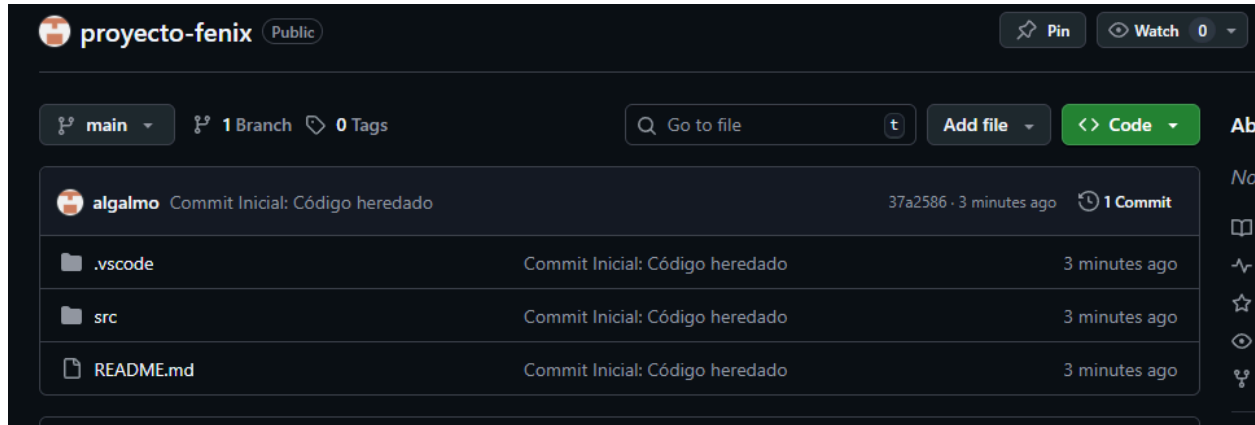


mensaje:

"Commit Inicial: Código heredado".

5. Se realiza push al repositorio remoto.





2. ANÁLISIS Y PRUEBAS

Tarea 2: Análisis estático de código

Se instala el plugin SonarQube en el IDE y se ejecuta el análisis sobre la clase `ProcesadorUsuarios.java`.

Problemas detectados por SonarQube:

- Uso de números mágicos en el código.
- Nombres de variables poco descriptivos.
- Método demasiado largo y con múltiples responsabilidades.

Se explora la sección SonarQube -> Rules para comprobar las reglas activas y su configuración.

PROBLEMS 6 OUTPUT DEBUG CONSOLE TERMINAL PORTS SONARQUBE 4

SONARQUBE FINDINGS (ALL FINDINGS)

ProcesadorUsuarios.java src\ 6

Move this file to a named package. (java:S1220) [Ln 1, Col 0]

Remove this unused import 'java.util.ArrayList'. (java:S1128) [Ln 1, Col 7]

Use a StringBuilder instead. (java:S1643) [Ln 22, Col 20]

Use a StringBuilder instead. (java:S1643) [Ln 26, Col 20]

Rules

▼ RULES Active All Inactive ...

Changes to this view are restricted to your personal development environment; to share a rule set with your team, please use Connected Mode.

> AzureResourceManager

> C

> C#

> C++

> CSS

> CloudFormation

> Docker

> Go

> HTML

> IPython Notebooks

> Java

> JavaScript

> Kubernetes

> PHP

> Python

> Secrets

> Terraform

> TypeScript

> XML

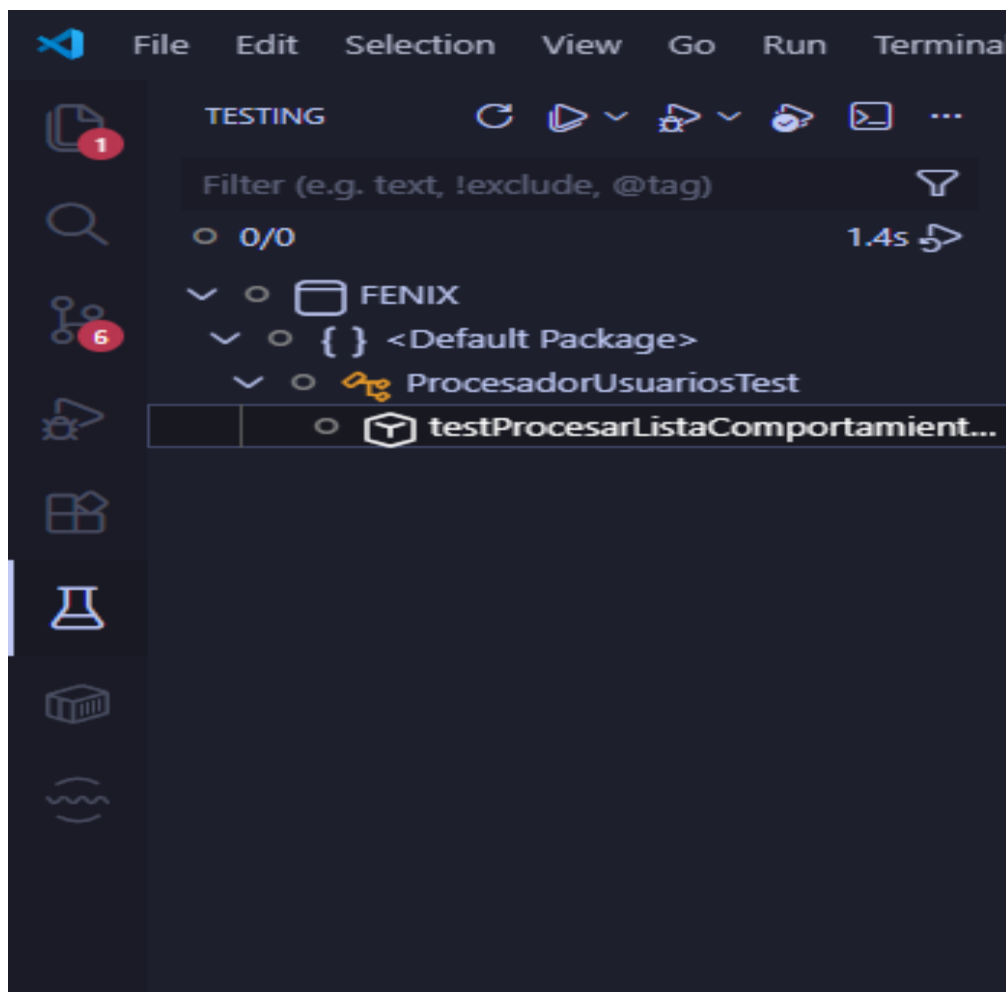
Tarea 3: Pruebas de refactorización

Antes de refactorizar, se crea una red de seguridad mediante pruebas unitarias con JUnit.

Se crea la clase `ProcesadorUsuariosTest.java` que comprueba el comportamiento actual del método.

La prueba pasa correctamente, confirmando que el comportamiento actual es válido.

Se realiza un commit con el mensaje "Añadidas pruebas de refactorización" y se hace push.



3. REFACTORIZACIÓN

Tarea 4: Aplicar patrones de refactorización

Con las pruebas como respaldo, se procede a refactorizar el código:

- Se sustituyen números mágicos por constantes.
- Se renombran variables con nombres significativos.
- Se extraen métodos para reducir la complejidad del método principal.

Tras la refactorización, se vuelven a ejecutar las pruebas unitarias, las cuales siguen pasando.

Se realiza un commit con el mensaje:

"Refactorización de ProcesadorUsuarios (Extract Method, Extract Constant)".

```
private static final int ROL_ADMIN = 1;
private static final int ROL_INVITADO = 2;

private String procesarAdmin(String nombre) { ... }

private String procesarInvitado(String nombre) { ... }
```

```
1
2 import static org.junit.jupiter.api.Assertions.assertEquals;
3 import java.util.List;
4 import org.junit.jupiter.api.Test;
5
6 public class ProcesadorUsuariosTest {
7
8     @Test
9     void testProcesarListaComportamientoActual() {
10         ProcesadorUsuarios procesador = new ProcesadorUsuarios();
11         List<String> lista = List.of("Ana:1", "Luis:2", "Eva:1", "Juan:99");
12
13         String resultado = procesador.procesarLista(lista);
14
15         assertEquals("Admins: Ana,Eva, | Invitados: Luis,", resultado);
16     }
17 }
```



4. DOCUMENTACIÓN

Tarea 5: Generar documentación

Se añaden comentarios Javadoc a la clase `ProcesadorUsuarios` y a sus métodos públicos, utilizando las etiquetas `@param` y `@return`.

Posteriormente se genera la documentación HTML desde el IDE.

```
1 // Importa las funciones necesarias para hacer comprobaciones (asserts)
2 import static org.junit.jupiter.api.Assertions.assertEquals;
3
4 // Importa List para crear listas de usuarios
5 import java.util.List;
6
7 // Importa la anotación @Test de JUnit 5
8 import org.junit.jupiter.api.Test;
9
10 /**
11  * Clase de pruebas unitarias para la clase ProcesadorUsuarios.
12  * Sirve como red de seguridad antes de refactorizar el código.
13  */
14 public class ProcesadorUsuariosTest {
15
16     /**
17      * Comprueba que el método procesarLista mantiene
18      * su comportamiento actual.
19      */
20     @Test
21     void testProcesarListaComportamientoActual() {
22
23         // Se crea una instancia de la clase a probar
24         ProcesadorUsuarios procesador = new ProcesadorUsuarios();
25
26         // Lista de entrada con el formato "nombre:rol"
27         // 1 = Admin, 2 = Invitado, otros valores se ignoran
28         List<String> lista = List.of(
29             "Ana:1",
30             "Luis:2",
31             "Eva:1",
32             "Juan:99"
33         );
34
35         // Se ejecuta el método que se quiere comprobar
36         String resultado = procesador.procesarLista(lista);
37
38         // Se compara el resultado obtenido con el esperado
39         assertEquals(
40             "Admins: Ana,Eva, | Invitados: Luis,",
41             resultado
42         );
43     }
44 }
45
```

```

src > index.html > ...
2 <html lang="es">
3 <head>
4 <style>
5 body {
6 }
7
8 h1 {
9 color: #2c3e50;
10 }
11
12 h2 {
13 color: #34495e;
14 }
15
16 .class-box {
17 border: 1px solid #ccc;
18 padding: 20px;
19 border-radius: 6px;
20 background-color: #f9f9f9;
21 }
22
23 .method {
24 margin-left: 20px;
25 }
26
27 code {
28 background-color: #eee;
29 padding: 3px 6px;
30 border-radius: 4px;
31 }
32 </style>
33 </head>
34 <body>
35
36 <h1>Proyecto Fénix - Documentación Javadoc</h1>
37
38 <div class="class-box">
39 <h2>Clase ProcesadorUsuarios</h2>
40
41 <p>
42 Esta clase se encarga de procesar una lista de usuarios,
43 clasificándolos según su rol (administrador o invitado).
44 </p>
45
46 <h3>Métodos</h3>
47
48 <div class="method">
49 <p>
50 <strong><code>String procesarLista(List<String> usuarios)</code></strong>
51 </p>
52 <p>
53 Procesa una lista de usuarios con formato <code>nombre:rol</code>
54 y devuelve una cadena con los administradores e invitados.
55 </p>
56 <p>
57 <strong>Parámetros:</strong><br>
58 <code>usuarios</code> Lista de usuarios a procesar
59 </p>
60 <p>
61 <strong>Devuelve:</strong><br>
62 Una cadena con los usuarios clasificados por rol
63 </p>
64 </div>
65 </div>
66
67 </body>
68 </html>

```

Se realiza un commit con el mensaje "Documentación Javadoc añadida".

5. INTEGRACIÓN CONTINUA

Tarea 6: Integración continua con GitHub Actions

Se activa GitHub Actions en el repositorio.

Se crea un workflow para Java con Maven que ejecuta automáticamente las pruebas JUnit mediante el comando mvn test.

Se realiza un cambio trivial en el código y se hace push para comprobar el funcionamiento.



CONCLUSIÓN

La actividad demuestra el uso completo de las herramientas del entorno de desarrollo para mejorar código heredado de forma segura, profesional y automatizada.